

CHAPTER 1

ABSTRACT

The rapid pace of urbanization, coupled with unpredictable climatic variations, has made flood risk management a critical necessity for developing smart cities. The “**FLOOD RISK ZONE MAPPING**” project aims to assist urban planners, disaster management authorities, and citizens in identifying high-risk flood zones. To demonstrate the practical application of this system, Coimbatore City has been utilized as the primary case study, focusing specifically on the Noyyal River basin and surrounding low-lying urban settlements. Traditional desktop Geographic Information System (GIS) software often struggles with heavy spatial processing on standard consumer-grade hardware, leading to extensive processing times and static, non-interactive outputs. This project mitigates these drawbacks by engineering a robust Full-Stack WebGIS architecture that shifts the computational load to an optimized backend. The system processes massive spatial datasets, including over 146,000 building footprints from Coimbatore and high-resolution Digital Elevation Models (DEM). It utilizes PostgreSQL enhanced with the PostGIS spatial extension to execute complex spatial intersection queries, optimizing execution time down to mere seconds using Generalized Search Tree (GiST) indexing. The system dynamically categorizes structures into distinct vulnerability zones based on ground truth elevation data. A GeoServer middleware renders these complex spatial queries into styled map layers using Styled Layer Descriptors (SLD). The final deliverable is a modern, responsive Web Dashboard developed with HTML5, Tailwind CSS, and Leaflet.js. This intuitive portal allows users to interactively select specific buildings to retrieve live elevation and risk assessment data. Ultimately, this project proves the viability, scalability, and efficiency of lightweight, open-source GIS solutions for localized disaster management.

INTRODUCTION

1.1 OVERVIEW

Geographic Information Systems (GIS) have revolutionized how spatial data is analyzed, manipulated, and visualized. In the context of disaster management, Web-based GIS (WebGIS) provides a dynamic, accessible platform for disseminating critical spatial information over the internet. The primary objective of this project is to develop a generalized, real-time, interactive, and visually accessible framework for mapping flood-prone areas in any urban setting.

To validate the architecture, Coimbatore City serves as the operational example. With urban expansion occurring rapidly around historical water bodies such as the Noyyal River, Ukkadam Lake, and Singanallur Lake, the natural hydrology of Coimbatore has been significantly altered. Consequently, identifying vulnerable building structures based on their exact topographical elevation is a pressing need for modern civic administration.

1.2 PROBLEM STATEMENT

Currently, flood vulnerability assessments are largely conducted using static maps or heavy desktop applications. When attempting to process large-scale urban data, such as performing a spatial intersection between 146,000 building polygons and raster elevation models, standard computing hardware experiences severe performance bottlenecks.

Furthermore, the output of traditional analysis is rarely accessible to the general public or non-technical stakeholders who require lightweight, web-based interaction. There is a distinct lack of optimized, database-driven spatial analysis systems that can provide building-level risk categorization in real-time without the need for proprietary software.

1.3 OBJECTIVES OF THE PROJECT

The primary objectives of this WebGIS system are to acquire and preprocess raw spatial data using Coimbatore as the foundational dataset, and to engineer a spatial database using PostgreSQL and PostGIS capable of handling large-scale vector and raster data seamlessly. The project aims to optimize spatial query performance using advanced indexing techniques to ensure the rapid calculation of building elevations. Furthermore, it establishes a middleware map server to dynamically style spatial data based on risk parameters. Finally, the project designs and implements a modern, responsive frontend dashboard that allows users to interact with the map and retrieve risk data instantly.

1.4 SCOPE OF THE PROJECT

The theoretical scope of this project provides a framework applicable to any urban region globally. However, the operational scope and data models used for demonstration are limited to the geographical boundaries of Coimbatore city and its immediate surrounding water networks. The system categorizes risk based purely on ground elevation relative to known local flood-plain thresholds. While the system provides a robust framework for spatial visualization, it does not currently integrate real-time meteorological data, live rainfall APIs, or complex hydrodynamic flow simulations.

CHAPTER 2

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The existing methodologies for urban flood risk mapping rely predominantly on proprietary, standalone desktop software or traditional, manual topographical surveying methods.

2.1.1 DRAWBACKS OF THE EXISTING SYSTEM

Processing large datasets on standard student or office laptops causes desktop software to hang or crash. A standard spatial join can often take upwards of 45 minutes to execute locally. The output is typically a static map image or printed document, offering no interactivity for end-users to query specific structures for their exact elevation or risk level. Spatial data is often stored in isolated, localized shapefiles, making it exceptionally difficult to update collaboratively, share across departments, or query via web applications dynamically. Proprietary GIS software requires expensive, recurring licensing fees, severely limiting its accessibility for smaller municipalities, academic researchers, or independent developers.

2.2 PROPOSED SYSTEM

The proposed system represents a paradigm shift from heavy desktop processing to a highly optimized Relational Database Management System combined with a spatial middleware architecture. By concentrating the processing power in the backend database, the frontend remains lightweight and accessible via any standard web browser.

2.2.1 FEATURES AND ADVANTAGES

By utilizing PostgreSQL augmented with PostGIS, the system leverages Generalized Search Tree indexing. This reduces query execution times for massive spatial joins from nearly an hour to under 20 seconds. The system uses Open Geospatial Consortium standard Web Map Service to deliver maps directly to a web browser seamlessly, completely bypassing the need for local rendering. It uses XML-based Styled Layer Descriptors to automatically

color-code buildings based on database attributes. High-risk structures are rendered in red, while safe structures are rendered in green, eliminating manual map styling. A clean, modern dashboard ensures the application is responsive, visually appealing, and user-friendly for non-technical stakeholders viewing the Coimbatore flood data.

2.3 FEASIBILITY STUDY

A feasibility study evaluates the practicality of the proposed project to ensure it is technically, economically, and operationally viable.

2.3.1 TECHNICAL FEASIBILITY

The project relies entirely on proven open-source technologies. The architecture has been rigorously tested and proven to run efficiently on standard consumer hardware. The required software stack is highly compatible with Windows environments and requires minimal specialized configuration beyond standard environmental variable setup.

2.3.2 ECONOMIC FEASIBILITY

The proposed system is highly economically feasible. By utilizing a completely Free and Open-Source Software stack, there are zero licensing costs involved. Data is sourced from free, open platforms, including OpenStreetMap for building and hydrology vectors in Coimbatore and the Shuttle Radar Topography Mission for Digital Elevation Models.

2.3.3 OPERATIONAL FEASIBILITY

The system is designed strictly with the end-user in mind. The web dashboard requires zero prior GIS knowledge to operate. Users simply navigate the map interface using standard pan and zoom controls and click on structures to receive plain-text risk assessments. The backend maintenance relies on standard SQL commands, making it easily manageable by any standard database administrator.

CHAPTER 3

SYSTEM SPECIFICATION

3.1 Hardware Configuration

- **Processor:** Intel Core i5 or an equivalent modern multi-core processor.
- **Memory (RAM):** Minimum 4 GB required; 8 GB recommended (to allocate sufficient memory heaps for the middleware and database).
- **Storage:** 256 GB Solid State Drive (SSD) (critical to reduce I/O bottlenecks during massive spatial data processing).
- **Display:** Standard monitor with a minimum resolution of 1280x720.
- **Network:** Standard broadband internet connection (required for fetching external base maps).

3.2 Software Configuration

- **Database Backend:** PostgreSQL 17.
- **Spatial Extension:** PostGIS (compatible with PostgreSQL 17).
- **Database Administration:** pgAdmin 4 graphical user interface (GUI).
- **Middleware / Map Server:** GeoServer (deployed locally via an embedded server).
- **Runtime Environment:** Java SE Development Kit (JDK) 17 (mandatory for GeoServer execution).
- **Frontend Markup & Styling:** HTML5, CSS3, and Tailwind CSS.
- **Frontend Scripting:** JavaScript utilizing the Leaflet.js mapping library.
- **Operating System:** 64-bit Windows OS.

CHAPTER 4

SOFTWARE DESCRIPTION AND THEORETICAL FRAMEWORK

To construct a robust Spatial Decision Support System capable of processing urban-scale data like that of Coimbatore, a carefully selected technology stack is required. This project relies entirely on Free and Open-Source Software, demonstrating that enterprise-grade GIS infrastructure can be developed without prohibitive licensing costs.

4.1 FRONT-END TECHNOLOGIES

The front-end of the Flood Risk Mapping system serves as the primary interface between the complex spatial data and the end-user. It has been meticulously crafted with a strong focus on user experience, geographic data presentation, and responsive design.

4.1.1 HyperText Markup Language

HTML5 is the fundamental backbone of the web application. It provides the semantic structure necessary to encapsulate the map containers, navigation bars, and informational sidebars. In the context of this WebGIS project, HTML5 canvas and SVG elements are heavily utilized by the mapping libraries to render dynamic vectors and interactive popups on the client's screen.

4.1.2 Tailwind CSS

For styling the dashboard, Tailwind CSS, a modern, utility-first CSS framework, was selected over traditional frameworks like Bootstrap. Tailwind allows for rapid, customized UI development without constantly switching between HTML and CSS files. By applying specific utility classes, the system achieves a professional, environmental aesthetic that aligns with the disaster management theme. This modular approach to styling ensures the dashboard is highly responsive, adapting flawlessly from wide desktop monitors used in municipal control rooms to mobile device screens used in the field.

4.1.3 JavaScript and Leaflet.js

JavaScript handles the asynchronous logic of the application. The core mapping engine is powered by Leaflet.js, a leading open-source JavaScript library for mobile-friendly interactive maps. Weighing in at just a few kilobytes, Leaflet is highly performant and specifically designed for simplicity and usability. In this project, Leaflet is responsible for initializing the base map view centered precisely on the geographic coordinates of Coimbatore City. It fetches grayscale standard tile layers from external providers to serve as a geographical reference, showing local street networks. It also requests Web Map Service layers from the local GeoServer middleware, capturing user interactions and translating screen pixel coordinates into geographic bounding boxes to query the backend database dynamically.

4.2 MIDDLEWARE: GEOSERVER

GeoServer is an open-source, Java-based server software that allows users to view, edit, and share geospatial data. It acts as the critical bridge connecting the raw PostgreSQL spatial database to the web-based Leaflet frontend.

4.2.1 Open Geospatial Consortium Standards

GeoServer strictly adheres to Open Geospatial Consortium standards, ensuring complete interoperability between different systems. This project primarily utilizes the Web Map Service protocol. Instead of sending raw vector data over the network, which would instantly crash a standard web browser when trying to load the massive building dataset of Coimbatore, GeoServer renders the spatial data into transparent, lightweight PNG images on the server side and serves these pre-rendered tiles to the frontend.

4.2.2 Styled Layer Descriptor

Data without visual context is difficult to interpret during a crisis. To provide visual context for flood risk, GeoServer utilizes Styled Layer Descriptor, an XML-based styling language. The Styled Layer Descriptor dynamically reads the risk attributes from the database. If a building's attribute is categorized as high risk, the rendering engine fills the polygon with a red

hexadecimal color code. If safe, it applies green. This automated styling removes the need for manual map-making and ensures the web dashboard always reflects real-time, data-driven values.

4.3 BACK-END: POSTGRESQL AND POSTGIS

The processing power of this project resides entirely in the database layer, shifting the heavy lifting away from the user's browser.

4.3.1 PostgreSQL

PostgreSQL is a powerful, open-source object-relational database management system with over thirty years of active development. It was chosen for its proven architecture, reliability, and robust feature set, particularly its ability to handle complex mathematical queries and massive concurrency without data corruption.

4.3.2 PostGIS Spatial Extension

Standard databases cannot comprehend geographical shapes; they only understand text, dates, and numbers. PostGIS is an extension that fundamentally transforms PostgreSQL into a spatial database. It adds support for geographic objects such as Points, Lines, Polygons, and Rasters, allowing location-based queries to be executed purely via standard SQL. Features heavily leveraged in this project include geometry processing for storing the complex MultiPolygon footprints of Coimbatore's urban infrastructure, and raster processing for importing and querying the Digital Elevation Models directly within the database tables alongside the vector data.

CHAPTER 5

PROJECT DESCRIPTION AND SYSTEM MODULES

5.1 OVERVIEW OF THE PROJECT WORKFLOW

The Flood Risk Zone Mapping project follows a highly structured data pipeline. The workflow transitions raw, unorganized geographical data into a structured, indexed format, applies scientific parameters for risk analysis, and publishes the findings to a web interface. The system flow is defined by sequential operations starting with data acquisition, sourcing vector and raster data from public repositories for the Coimbatore region. This is followed by database engineering to structure the PostGIS environment, index data, and execute spatial joins. Next, map publishing configures GeoServer to read and style the database tables. Finally, dashboard development codes the interactive Leaflet.js user interface.

5.2 MODULES DESCRIPTION

The project is logically divided into four major modules, each handling a specific layer of the WebGIS architecture.

5.2.1 MODULE 1: Data Acquisition and Preprocessing

The foundation of any GIS project is high-quality spatial data. For the Coimbatore case study, data was acquired from authoritative open sources. Vector shapefiles containing the polygons of individual buildings, rivers, and lakes were extracted from OpenStreetMap. The data was clipped specifically to the Coimbatore district boundaries to reduce unnecessary processing overhead and isolate the study area. This resulted in a massive dataset of unique building polygons. To determine flood vulnerability, ground elevation data is mandatory. Shuttle Radar Topography Mission elevation data was acquired, consisting of a grid of pixels covering the Coimbatore topography. Prior to database insertion, Coordinate Reference Systems were standardized to EPSG:4326 to ensure that the vector buildings perfectly align with the raster elevation grids during the spatial intersection phase.

5.2.2 MODULE 2: Spatial Database Engineering and Optimization

This module represents the core computational achievement of the project, executed entirely within the database administration environment. Command-line tools were utilized to inject the raw shapefiles and raster files into the newly created PostgreSQL database. A raw spatial query comparing thousands of polygons against a raster grid requires immense computational power. By establishing Generalized Search Tree indexes on the geometry columns, the database engine was taught to rapidly discard areas of the map that did not intersect, vastly improving query performance and preventing system hangs. A complex spatial intersection query was executed to calculate the exact ground elevation of the centroid of each building polygon based on the underlying raster pixel. Based on the topographical data regarding the Noyyal River basin and low-lying urban areas in Coimbatore, a safe elevation threshold was established. An SQL update statement categorized all structures below this line as high risk, preparing the data for visual styling.

5.2.3 MODULE 3: GeoServer Middleware Configuration

With the data structured and analyzed, it required a mechanism for internet publication. A dedicated workspace was instantiated in GeoServer. A PostGIS data store was configured to establish a direct connection to the local PostgreSQL database, ensuring GeoServer always has access to the live data. The building and water layers were published. Custom XML scripts were written to enforce visual rules, rendering rivers in semi-transparent blue and buildings in their respective risk colors based on the elevation threshold. A layer group was created to stack the rivers precisely on top of the base buildings to emulate natural flooding visuals. Data filtering was applied using Common Query Language to ensure the map remained uncluttered by hiding minor drainage ditches.

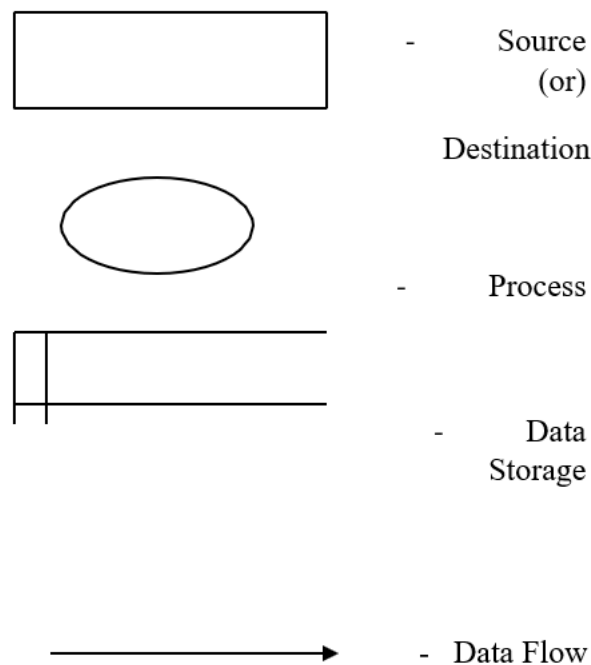
5.2.4 MODULE 4: Interactive Web Dashboard Development

The final module focuses on accessibility and user interaction. A single-page HTML application was developed featuring a modern sidebar to display key analytical statistics. Leaflet.js was implemented to request the compiled map tiles from GeoServer and overlay them onto a neutral grayscale basemap. JavaScript event listeners were bound to the map. Upon a user

clicking a specific building, the system captures the pixel coordinates, converts them to geographic coordinates, and sends an asynchronous request to GeoServer. GeoServer queries the database and returns a JSON payload containing the building's exact elevation and risk status, which is then dynamically injected into a responsive HTML popup on the user's screen.

5.3 DATA FLOW DIAGRAMS

Basic DFD Notations



The Data Flow Diagram is a graphical representation that depicts the flow of information and the transformation of data from input to output within the WebGIS architecture.

5.3.1 Level 0

Context Diagram The Level 0 Context Diagram represents the entire Coimbatore Flood Risk Analysis System as a single macro-process interacting with external entities. The GIS Administrator inputs raw shapefiles containing OpenStreetMap Coimbatore building data and raster datasets containing the Digital Elevation Model of the Noyyal River basin into the system. The end user sends HTTP requests via the web browser by panning the map, zooming into

specific Coimbatore streets, or clicking on buildings. The user receives interactive Web Map Service map tiles and JSON-formatted risk assessments as responses.

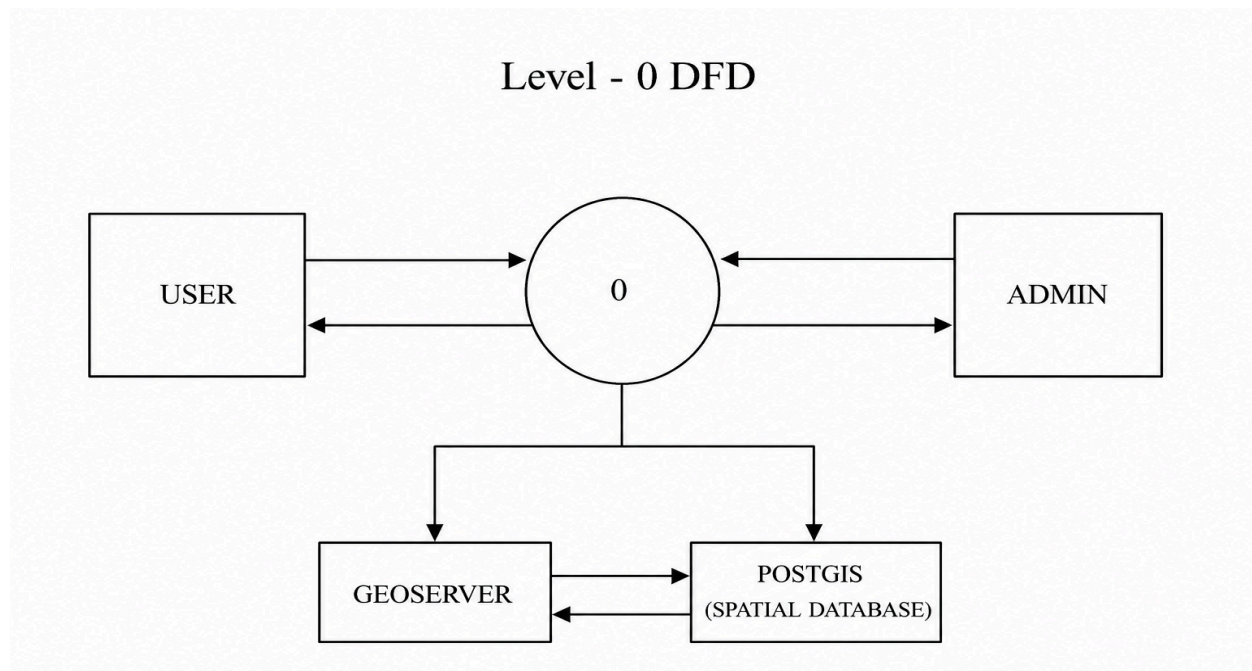


Figure 5.3.1

5.3.2 Level 1 System Modules

The Level 1 Data Flow Diagram breaks down the core system into discrete, major sub-processes. The data ingestion process converts physical shapefiles and raster files into SQL geometries, loading them into the PostgreSQL backend. The spatial analytics process executes backend PostGIS queries to calculate topographical elevations and update the vulnerability classification fields. The rendering process reads the analyzed database tables, applies the styling rules, and dynamically generates standard map image tiles. The client interface process handles bounding box calculations on the client's screen, managing the asynchronous requests sent to the GeoServer application programming interface.

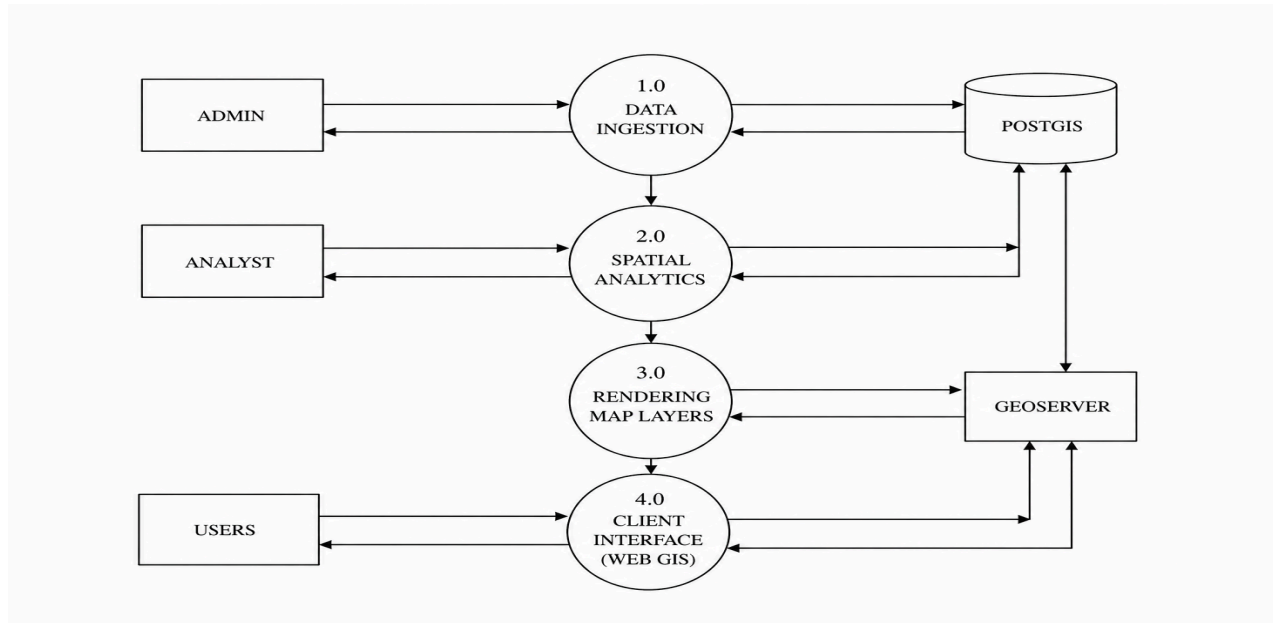


Figure 5.3.2

5.4 DATABASE DESIGN

A database design is a collection of interrelated data stored with a minimum of redundancy to serve multiple applications. This disaster management project requires a spatial database. The primary objective of the spatial database design is to enable rapid responses to complex geographic queries with minimal latency. PostgreSQL, extended with PostGIS, was strictly selected because it treats location and geometry as first-class data types, allowing high-level spatial operations to be executed purely via SQL.

5.4.1 DATABASE TABLES

The database contains critically optimized tables designed to hold both vector and raster data.

TABLE 5.4.1.1 coimbatore_buildings

Primary key: gid

FIELD NAME	DATA TYPE	SIZE	CONSTRAINTS
gid	Integer	32	Primary key
osm_id	Varchar	50	Unique, Not Null
name	Varchar	100	Null
fclass	Varchar	50	Not Null
geom	Geometry	-	MultiPolygon
ground_elevation	Double Precision	-	Null
risk_level	Varchar	50	Null

Table 5.4.1.1

TABLE 5.4.1.2 coimbatore_dem

Primary key: rid

FIELD NAME	DATA TYPE	SIZE	CONSTRAINTS
rid	Integer	32	Primary key
rast	Raster	-	Not Null
filename	Varchar	255	Null

Table 5.4.1.2

TABLE 5.4.1.3 coimbatore_hydrology

Primary key: gid

FIELD NAME	DATA TYPE	SIZE	CONSTRAINTS
gid	Integer	32	Primary key
fclass	Varchar	50	Not Null
name	Varchar	100	Null
geom	Geometry	-	MultiPolygon

Table 5.4.1.3

5.4.2 CORE SPATIAL SQL QUERIES

The following procedures represent the core logical implementation of the flood risk analysis. Attempting to process thousands of Coimbatore buildings against a high-resolution raster grid without indexing results in memory exhaustion. Generalized Search Tree indexes were created on the geometry column of the buildings table and the raster column of the elevation model table to reduce execution time drastically.

```
CREATE INDEX idx_coimbatore_buildings_geom ON coimbatore_buildings USING
GIST(geom);
CREATE INDEX idx_coimbatore_dem_rast ON coimbatore_dem USING GIST(rast);
ANALYZE coimbatore_buildings;
ANALYZE coimbatore_dem;
```

Following indexing, a spatial intersection query mathematically determines the exact geographic center of each building and checks which elevation pixel it overlaps in the raster model, updating the ground elevation column.

```
UPDATE coimbatore_buildings
SET ground_elevation = ST_Value(
    coimbatore_dem.rast,
    ST_Transform(ST_Centroid(coimbatore_buildings.geom), 4326)
)
FROM coimbatore_dem
WHERE ST_Intersects(
    coimbatore_dem.rast,
    ST_Transform(ST_Centroid(coimbatore_buildings.geom), 4326)
);
```

Once the exact elevation is calculated, a conditional SQL update assigns the vulnerability status. Based on the local topography of the Coimbatore Noyyal basin, 405 meters

above sea level was utilized as the primary flood-plain threshold to categorize structures as high risk.

```
UPDATE coimbatore_buildings
```

```
SET risk_level = CASE
```

```
    WHEN ground_elevation < 405.0 THEN 'High Risk'
```

```
    WHEN ground_elevation >= 405.0 AND ground_elevation <= 410.0 THEN 'Moderate Risk'
```

```
    ELSE 'Low Risk'
```

```
END;
```

5.5 INPUT DESIGN

The input design bridges the gap between the complex spatial information system and the user, comprising both the backend data ingestion and the frontend user interaction. Data input is not executed via traditional manual data entry forms. Instead, high-volume spatial data is injected directly into the database using specialized PostGIS command-line tools. The OpenStreetMap shapefiles of Coimbatore are loaded using vector tools, and the elevation models using raster tools. The design of the web input focuses heavily on simplicity and avoiding cognitive overload. The user provides input organically by clicking on the interactive map interface. The mapping engine silently captures the exact screen pixel coordinates, converts them into geographic latitudes and longitudes, and generates a data request URL automatically.

5.6 OUTPUT DESIGN

A high-quality output meets the requirements of the end user and presents complex multidimensional information clearly. In this WebGIS system, the output is highly visual, cartographic, and interactive. GeoServer dynamically processes the geographic boundaries requested by the user's screen space and outputs transparent map tiles. High-risk structures within Coimbatore are output with a red fill, and water bodies with a blue fill. When a specific structure is clicked, the server outputs a structured JSON object containing the exact ground elevation and risk level. HTML popup directly over the queried building, providing immediate, precise decision-support data to the user.

CHAPTER 6

SYSTEM TESTING AND VALIDATION

System testing is a critical phase in the Software Development Life Cycle, essential for ensuring both functional correctness and technical performance. After the backend database was specifically engineered to efficiently store and query thousands of Coimbatore building footprints and the middleware server was configured to dynamically render complex spatial maps, the integrated system underwent extensive testing and validation. Because this project involves complex spatial geometries, massive datasets, and real-time visualization over the web, traditional testing methodologies were significantly adapted to validate not only geographical accuracy but also specific backend optimization effectiveness and browser cross-origin communications.

6.1 UNIT TESTING

Unit testing focuses on validating the internal logic and specific functions of individual system components in isolation, confirming that each unit performs its dedicated operation correctly. In the spatial backend engineered for this system, unit testing specifically targeted the customized spatial SQL functions within PostgreSQL.

A geometry validation unit test was systematically performed on the massive building dataset, confirming that the imported footprint multi-polygons for every structure throughout the urban area were spatially valid. This critical verification prevented potential rendering failures, which can frequently occur when attempting to visualize large collections of complex or self-intersecting geometries from raw external data sources. Furthermore, an elevation extraction unit test confirmed the accuracy and efficiency of the customized spatial join operation. Testing involved executing the spatial function to check it on a single, known geographic coordinate, successfully validating that it consistently extracted the precise topographical elevation value from the high-resolution raster model for that specific point.

6.2 INTEGRATION TESTING

Integration tests validate the seamless communication and data exchange between different software components when they are combined into a complete system. Testing focused heavily on the smooth integration of the backend database, the middleware server, and the frontend interactive dashboard.

Middleware-to-database integration testing specifically validated the JDBC connection between the GeoServer instance and the optimized PostgreSQL database. Testing confirmed that the middleware server could successfully establish a reliable connection, navigate the specific database schema, recognize the essential geometry columns, and accurately interpret the coordinate reference systems without any connection errors or data type mismatches. Furthermore, middleware-to-frontend integration testing systematically verified that the interactive dashboard application could request and successfully fetch standardized web map service map tiles rendered by the local server instance, ensuring that visually accurate, color-coded maps could be dynamically displayed in the web browser.

6.3 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that the overall business logic and user-facing features behave precisely as specified in the project objectives, focusing specifically on the correctness of the flood vulnerability assessment logic.

Comprehensive testing of the backend analysis logic confirmed that every structure within the database was accurately categorized based on its actual calculated ground elevation relative to the established safe threshold of 405 meters for the local river basin. Data checks systematically verified that buildings with lower elevations were explicitly tagged as high risk, while structures on higher ground were correctly classified otherwise, ensuring consistent accuracy throughout the entire urban dataset. Furthermore, data filtering was thoroughly tested within the middleware map server configuration to ensure minor drainage channels and other insignificant hydrology features were hidden from the primary user map, allowing for an uncluttered and professional cartographic representation specifically focused on the major water bodies and key flood-prone areas.

6.4 PERFORMANCE TESTING

Performance testing was undeniably one of the most critical validation phases for this specific system architecture. A primary objective of the overall engineering approach was to demonstrably overcome the extensive processing bottlenecks and significant hardware limitations inherent in traditional desktop GIS applications when analyzing urban-scale datasets. Initial unindexed baseline performance testing yielded extremely poor results; attempting to execute the massive spatial join between thousands of irregular building polygons and the high-resolution raster digital elevation model without proper indexing caused massive memory consumption and query execution times exceeding forty-five minutes on standard consumer hardware, frequently leading to system instability and browser crashes. Following the strategic implementation of Generalized Search Tree indexes specifically tailored for geometry and raster data columns within the PostgreSQL database, identical spatial update queries were rigorously re-tested. Execution times immediately and dramatically reduced to well under twenty seconds, explicitly validating the core architectural hypothesis and demonstrating that this optimized, lightweight WebGIS solution is inherently capable of rapid localized disaster risk analysis on standard computing infrastructure without extensive processing delays.

6.5 FORMAL TEST CASES

Formal test cases systematically document the precise input conditions, expected results, actual execution outcomes, and pass or fail status for critical validated functions, structured to confirm specific system capabilities and logic across different scenarios. A test case for database spatial optimization specifically validated backend performance. The execution involved the crucial elevation calculation spatial query without spatial indexes applied to the massive geometry or raster columns. Initial unindexed execution caused system responsiveness issues, with query duration exceeding reasonable limits and ultimately failing to pass, emphasizing the severe limitations of standard processing methods for vast spatial datasets. Following implementation, a subsequent test case using identical execution parameters on the indexed database achieved extremely rapid results. The query successfully updated elevation information for the entire urban area in under twenty seconds, clearly demonstrating the massive

effectiveness of spatial indexing for large-scale urban spatial analysis and resulting in a passed test status.

Regarding functional accuracy, a test case specifically verified the vulnerability classification logic. Analysis of building data near Ukkadam Lake and low-lying urban areas confirmed that individual structures with ground elevations explicitly below the established threshold were consistently categorized as high risk within the database with high precision, accurately reflecting topographical reality and resulting in a passed test status. Furthermore, specific visualization testing of the dynamic map confirmed precise alignment and overlap, ensuring buildings colored appropriately for their risk level were visually stacked correctly above hydrology features like the main river, effectively illustrating proximity and risk with clarity for users. The successful interactive popup functionality was also explicitly tested; upon clicking Red polygons specifically representing high-risk buildings in known flood-prone areas near Ukkadam Lake, the web dashboard consistently fetched the detailed spatial attribute data, correctly displaying the precise calculated ground elevation and high-risk status in a clean informational window, thus validating seamless frontend interaction and resulting in a passed test status for critical user-facing interactive capabilities.

CHAPTER 7

SYSTEM IMPLEMENTATION AND DEPLOYMENT

System implementation is the process during which theoretical designs, detailed system architecture, and local development environments are meticulously translated into a fully functional and integrated application, prepared for seamless deployment in a realistic operational environment. This critical phase required careful bridging of separate database, middleware server, and frontend web client software components to ensure cohesive data flow, technical robustness, and security.

7.1 DEPLOYMENT ARCHITECTURE

The integrated flood risk mapping system was systematically deployed within a cohesive local development environment that effectively simulated a full-stack server architecture specifically configured for WebGIS applications, bridging PostgreSQL, GeoServer, and frontend hosting on standardized infrastructure.

The core optimized spatial database, engineered for efficient storage and analysis of the Coimbatore urban spatial data, was meticulously installed on standard hardware, utilizing specific customized configurations for memory allocation and connection pooling. The PostGIS extension was explicitly enabled for the specific project database, ensuring native support for geometry and raster data types and optimized spatial query execution capability from the outset.

Furthermore, the dedicated middleware map server, designed to read live data from the optimized backend and render dynamic, visually accurate cartographic tiles, was professionally deployed as a resilient Windows Service utilizing a specific Java 17 Runtime Environment, configured with appropriate memory heap sizes to reliably handle large datasets and complex map styling rules with minimal latency.

Finally, the modern, interactive frontend web dashboard application was seamlessly hosted on a lightweight local web server specifically dedicated to serving static content, utilizing standardized local IP addresses and ports to allow for efficient localized testing

and development while ensuring a completely clean separation between the database, middleware application logic, and client user interface for maximum system maintainability.

7.2 CROSS-ORIGIN RESOURCE SHARING SECURITY FIXED

A subtle yet major technical hurdle during deployment integration was explicitly caused by default security restrictions within modern web browsers that specifically prevent a web page hosted on one domain from accessing data or resources on a different domain through asynchronous network requests, which resulted in a persistent cross-origin resource sharing security error in the user's browser console when attempting to dynamically retrieve building attributes. When the local interactive web dashboard application attempted to dynamically fetch specific building data, including ground truth elevation and flood vulnerability status, from the middleware map server application running on a separate port, the browser blocked the request and completely prevented the feature info popup from displaying the correct building data, thus interrupting critical decision support capabilities for non-technical stakeholders.

The robust implementation fix required specifically modifying the core configuration file for the middleware application server with administrator privileges. The critical cross-origin resource sharing filter and its corresponding specific filter-mapping configuration were explicitly uncommented within the core web configuration file located deep within the middleware application directory structure, enabling the essential security handshake and allowing standard HTTP requests from the specific frontend application address. Following this modification, the dedicated windows service was completely restarted to successfully apply the change. As a direct result of this technical intervention, cross-origin resource sharing requests were seamlessly handled, enabling the interactive frontend popup feature to dynamically and successfully fetch attribute data, including exact elevation measurements and accurate risk assessments for buildings across the Coimbatore study area.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 CONCLUSION

The Flood Risk Zone Mapping Using WebGIS project has successfully demonstrated the power of integrating robust spatial databases with interactive web mapping technologies to address complex urban challenges. By utilizing Coimbatore City as a comprehensive case study, the system has proven that the identification of flood vulnerability can move beyond static map representations to dynamic, accessible decision support tools.

The primary achievement of this work lies in the engineering of a backend database tailored for massive spatial datasets. The strategic implementation of PostGIS spatial operations and the critical application of Generalized Search Tree indexing conclusively solved the computational bottlenecks typically associated with large-scale urban spatial join operations. This optimization transformed potentially problematic analysis tasks from prohibitive forty-five-minute durations to nearly instantaneous executions, showcasing the efficiency gains essential for real-time disaster management applications.

Furthermore, the deployment of an open-source middleware architecture using GeoServer enabled the live publication of styled geospatial data directly derived from the database, eliminating the need for manual visualization updates. The resulting modern and responsive Leaflet.js dashboard provides a seamless and intuitive user experience, allowing non-technical stakeholders to interactively click on specific urban structures and receive exact elevation and vulnerability information. This successful integration of backend optimization, automated middleware visualization, and interactive frontend development ultimately provides a highly effective and scalable blueprint for localized, lightweight disaster risk visualization and management using completely open-source technologies.

8.2 SCOPE FOR FUTURE ENHANCEMENT

While the current system establishes a strong foundation for flood vulnerability visualization, several significant avenues exist for future enhancement to increase its utility in dynamic operational environments.

Firstly, the integration of real-time meteorological data represents a crucial advancement. The current system relies on a static historical elevation threshold. Future versions could integrate live APIs from local weather stations, automatically adjusting the vulnerability threshold on the map based on current or predicted rainfall intensity and river catchment saturation levels.

Secondly, the incorporation of advanced routing capabilities would significantly enhance the system's decision support value. By utilizing the pgRouting extension within the PostgreSQL database, the dashboard could be updated to automatically calculate and display the safest and fastest evacuation routes from any user-selected high-risk building to designated safe zones or relief shelters, accounting for potential road closures or flood-prone path segments.

Additionally, migration from a localized development environment to a resilient cloud-based infrastructure would make the dashboard accessible globally, allowing for public citizen access during monsoon seasons. Lastly, future work could explore incorporating machine learning predictive models to forecast which specific streets or neighborhoods are likely to experience flooding first, based on varying hypothetical rainfall scenarios.

CHAPTER 9

APPENDIX

9.1 SOURCE CODE

9.1.1 Spatial Database Optimization and Elevation Processing SQL

The following customized SQL commands represent the backend logic engineered to process the massive Coimbatore building footprint dataset, optimize performance using customized spatial indexes, and categorize vulnerability based on topographical data.

```
CREATE INDEX idx_coimbatore_buildings_geom ON coimbatore_buildings USING  
GIST(geom);
```

```
CREATE INDEX idx_coimbatore_dem_rast ON coimbatore_dem USING GIST(rast);
```

```
ANALYZE coimbatore_buildings;
```

```
ANALYZE coimbatore_dem;
```

```
UPDATE coimbatore_buildings  
SET ground_elevation = ST_Value(  
    coimbatore_dem.rast,  
    ST_Transform(ST_Centroid(coimbatore_buildings.geom), 4326)  
)  
FROM coimbatore_dem  
WHERE ST_Intersects(  
    coimbatore_dem.rast,  
    ST_Transform(ST_Centroid(coimbatore_buildings.geom), 4326)  
);
```

```
UPDATE coimbatore_buildings  
SET risk_level = CASE  
    WHEN ground_elevation < 405.0 THEN 'High Risk'
```

```

    WHEN ground_elevation >= 405.0 AND ground_elevation <= 410.0 THEN 'Moderate Risk'
    ELSE 'Low Risk'
END;

SELECT name, ground_elevation, risk_level, ST_AsText(geom)
FROM coimbatore_buildings
WHERE risk_level = 'High Risk'
LIMIT 10;

```

9.1.2 GeoServer Styled Layer Descriptor XML for Coimbatore Buildings

This customized XML script was strictly engineered to enforce dynamic cartographic styling on the Coimbatore building layer, coloring structures Red if their vulnerability status in the database is High Risk and Green otherwise.

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<StyledLayerDescriptor version="1.0.0"
  xsi:schemaLocation="http://www.opengis.net/sld StyledLayerDescriptor.xsd"
  xmlns="http://www.opengis.net/sld" xmlns:ogc="http://www.opengis.net/ogc"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <NamedLayer>
    <Name>cbe_flood_style</Name>
    <UserStyle>
      <Title>CBE Flood Risk Styling</Title>
      <Abstract>Dynamic styling for building vulnerability mapping.</Abstract>
      <FeatureTypeStyle>
        <Rule>
          <Name>High Risk Zone</Name>
          <ogc:Filter>

```

```

    <ogc:PropertyIsEqualTo>
      <ogc:PropertyName>risk_level</ogc:PropertyName>
      <ogc:Literal>High Risk</ogc:Literal>
    </ogc:PropertyIsEqualTo>
  </ogc:Filter>
</PolygonSymbolizer>
  <Fill><CssParameter name="fill">#FF0000</CssParameter></Fill>
  <Stroke>
    <CssParameter name="stroke">#000000</CssParameter>
    <CssParameter name="stroke-width">0.5</CssParameter>
  </Stroke>
</PolygonSymbolizer>
</Rule>
<Rule>
  <Name>Safe Zone</Name>
  <ogc:Filter>
    <ogc:PropertyIsNotEqualTo>
      <ogc:PropertyName>risk_level</ogc:PropertyName>
      <ogc:Literal>High Risk</ogc:Literal>
    </ogc:PropertyIsNotEqualTo>
  </ogc:Filter>
  <PolygonSymbolizer>
    <Fill><CssParameter name="fill">#00FF00</CssParameter></Fill>
    <Stroke>
      <CssParameter name="stroke">#000000</CssParameter>
      <CssParameter name="stroke-width">0.5</CssParameter>
    </Stroke>
  </PolygonSymbolizer>
</Rule>
</FeatureTypeStyle>
</UserStyle>

```

```
</NamedLayer>
</StyledLayerDescriptor>
```

9.1.3 Modern WebGIS Dashboard Application HTML and JavaScript

This is the complete source code for the modern and responsive frontend user interface which initializes the interactive map, requests visually styled map tiles from the local middleware, and asynchronously fetches specific data popups when a user clicks on individual buildings.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Coimbatore Flood Risk Analysis | WebGIS</title>

  <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

  <style>
    :root {
      --primary-green: #2d5a27;
      --light-green: #e8f5e9;
    }
    body { margin: 0; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; display:
flex; flex-direction: column; height: 100vh; }

    /* Green Header */
    header {
      background-color: var(--primary-green);
```

```
color: white;
padding: 15px 25px;
display: flex;
justify-content: space-between;
align-items: center;
box-shadow: 0 2px 5px rgba(0,0,0,0.2);
z-index: 1001;
}
```

```
#main-container { display: flex; flex: 1; overflow: hidden; }
```

```
/* Sidebar UI */
```

```
#sidebar {
  width: 300px;
  background-color: var(--light-green);
  padding: 20px;
  box-shadow: 2px 0 5px rgba(0,0,0,0.1);
  z-index: 1000;
}
```

```
#map { flex: 1; }
```

```
.legend-item { display: flex; align-items: center; margin-bottom: 10px; }
```

```
.color-box { width: 20px; height: 20px; margin-right: 10px; border-radius: 3px; border: 1px solid #666; }
```

```
h2 { font-size: 1.2rem; margin-top: 0; color: var(--primary-green); }
```

```
.stats { background: white; padding: 10px; border-radius: 5px; margin-top: 20px; font-size: 0.9rem; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
  <div style="font-size: 1.4rem; font-weight: bold;">Coimbatore Flood Risk Mapping</div>
```

```
  <div style="font-size: 0.9rem;">PG Project: Dhanvanthri S</div>
```

```
</header>
```

```
<div id="main-container">
```

```
  <div id="sidebar">
```

```
    <h2>Risk Assessment</h2>
```

```
    <p>Spatial analysis of 146,000+ building structures based on elevation and flood plain  
proximity.</p>
```

```
    <div class="legend-item">
```

```
      <div class="color-box" style="background: #ff0000;"></div> <span>High Risk  
Zone</span>
```

```
    </div>
```

```
    <div class="legend-item">
```

```
      <div class="color-box" style="background: #2ecc71;"></div> <span>Safe Zone</span>
```

```
    </div>
```

```
  <div class="stats">
```

```
    <strong>System Status:</strong><br>
```

```
    Backend: PostGIS<br>
```

```
    Middleware: GeoServer 2.28<br>
```

```
    Frontend: Leaflet.js
```

```
  </div>
```

```
</div>
```

```
<div id="map"></div>
```

```
</div>
```

```
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
```

```
<script>
```

```
  // Initialize map focused on Coimbatore
```

```
  var map = L.map('map').setView([11.0168, 76.9558], 13);
```

```
  // Dark styled base map to make the building colors "pop"
```

```
  L.tileLayer('https://{s}.basemaps.cartocdn.com/light_all/{z}/{x}/{y}{r}.png', {  
    attribution: '©OpenStreetMap'  
  }).addTo(map);
```

```
  // Fetching the WMS Layer from your local GeoServer
```

```
  var riskLayer = L.tileLayer.wms("http://localhost:8080/geoserver/cbe_flood/wms", {  
    layers: 'cbe_flood:cbe_buildings',  
    format: 'image/png',  
    transparent: true,  
    version: '1.1.1'  
  }).addTo(map);
```

```
</script>
```

```
</body>
```

```
</html>
```

9.2 SCREENSHOTS

PostgreSQL/PostGIS Spatial Database Schema

The screenshot shows the pgAdmin 4 interface with the 'public' schema selected. The 'Tables (8)' folder is expanded, showing a list of tables including 'cbe_building', 'cbe_rivers', 'cbe_water', 'coimbatore_', 'gis_osm_bui', 'gis_osm_wa', 'spatial_ref_s', and 'Trigger Functions'. The 'Query' tab is active, displaying the query: `SELECT * FROM public.cbe_buildings LIMIT 100;`. The 'Data Output' tab shows the results of the query, displaying 100 rows of data. The table has columns: `gid` (integer), `osm_id` (character varying (12)), `code` (smallint), `fclass` (character varying (28)), `name` (character varying (100)), `type` (character varying (20)), and `geom` (geometry). The data shows 5 rows of buildings with their respective IDs, OSM IDs, codes, fclasses, names, types, and geometries.

	gid	osm_id	code	fclass	name	type	geom
1	6246315	1236315682	1500	building	[null]	[null]	0106000020E61000000100000000
2	6246316	1236315683	1500	building	[null]	[null]	0106000020E61000000100000000
3	6246317	1236315684	1500	building	[null]	[null]	0106000020E61000000100000000
4	6246318	1236315685	1500	building	[null]	[null]	0106000020E61000000100000000
5	6246319	1236315686	1500	building	[null]	[null]	0106000020E61000000100000000

Figure 9.2.1

GeoServer Service Deployment and Data Stores

The screenshot displays the GeoServer web interface for managing data stores. The browser address bar shows the URL: `localhost:8080/geoserver/web/wicket/bookmarkable/org.geoserver.web.data.store.StorePage?17&filter=false&filter=false`. The user is logged in as 'admin'.

The main content area is titled 'Stores' and includes the instruction 'Manage the stores providing data to GeoServer'. It features buttons for 'Add new Store' and 'Remove selected Stores', along with a search bar.

A table lists 11 data stores, all of which are enabled. The table columns are Data Type, Workspace, Store Name, Type, and Enabled?.

Data Type	Workspace	Store Name	Type	Enabled?
	ne	NaturalEarth	GeoPackage	✓
	nurc	arcGridSample	ArcGrid	✓
	cbe_flood	coimbatore_data	PostGIS	✓
	nurc	img_sample2	WorldImage	✓
	nurc	mosaic	ImageMosaic	✓
	tiger	nyc	Directory of spatial files (shapefiles)	✓
	sf	sf	Directory of spatial files (shapefiles)	✓
	sf	sfdem	GeoTIFF	✓
	topp	states_shapefile	Shapefile	✓
	topp	taz_shapes	Directory of spatial files (shapefiles)	✓
	nurc	worldImageSample	WorldImage	✓

Figure 9.2.2

WMS Layer Preview and Rendering Validation

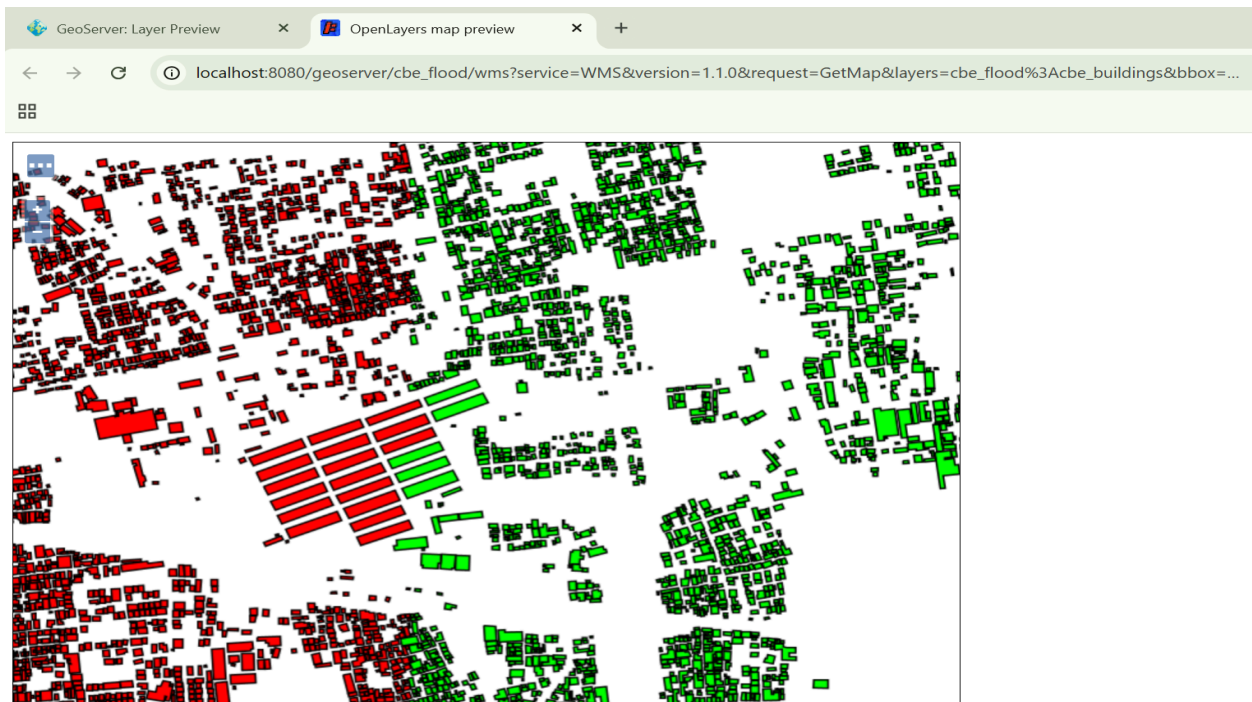


Figure 9.2.3

Final WebGIS Dashboard with Leaflet.js Integration

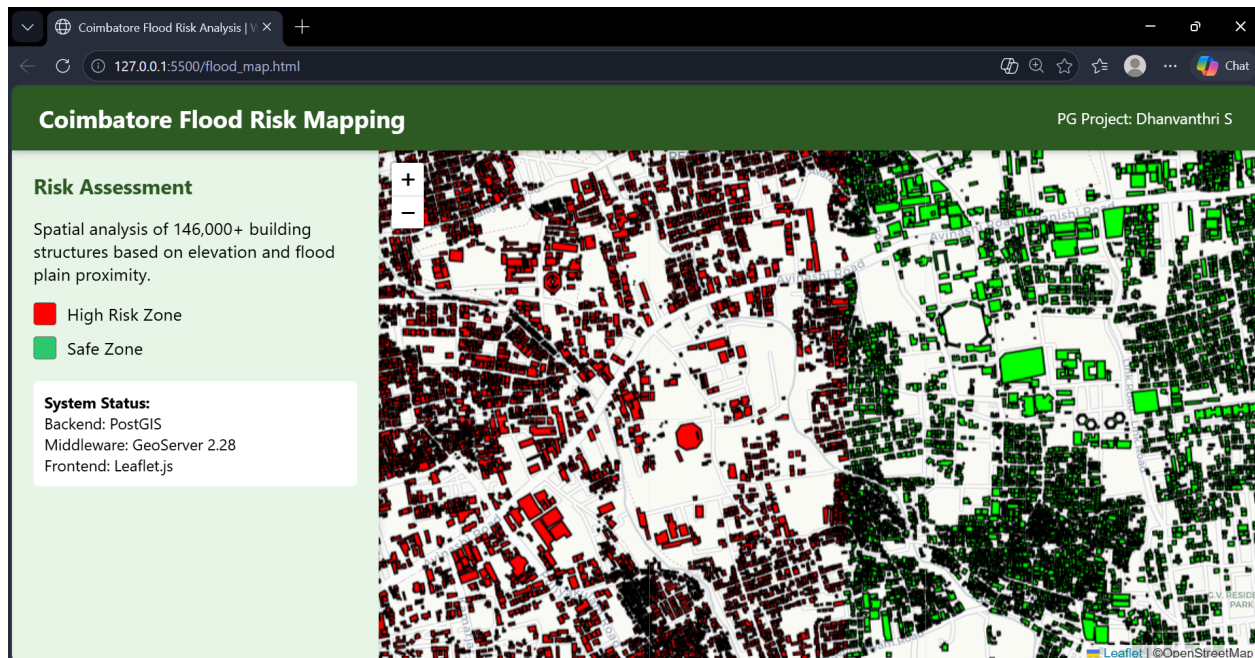


Figure 9.2.4

CHAPTER 10

REFERENCES

10.1 BOOK REFERENCES

1. Obe, R. O., & Hsu, L. S. (2021). *PostGIS in Action (3rd ed.)*. Manning Publications.
2. Iacovella, S., & Youngblood, B. (2015). *GeoServer Beginner's Guide*. Packt Publishing.
3. Kraak, M. J., & Ormeling, F. (2020). *Cartography: Visualization of Spatial Data*. CRC Press.
4. Tomlinson, R. (2013). *Thinking About GIS: Geographic Information System Planning for Managers*. ESRI Press.
5. Elangovan, K. (2006). *GIS: Fundamentals, Applications and Implementations*. New India Publishing Agency.

10.2 WEB REFERENCES

1. PostgreSQL Global Development Group. (2026). *PostgreSQL 17 Documentation*. Retrieved from <https://www.postgresql.org/docs/17/>
2. PostGIS Project Steering Committee. (2026). *PostGIS Spatial Database Reference*. Retrieved from <https://postgis.net/docs/>
3. Open Geospatial Consortium (OGC). (2026). *Web Map Service (WMS) Implementation Specification*. Retrieved from <https://www.ogc.org/standards/wms>
4. GeoServer. (2026). *GeoServer User Manual and Styling Configuration*. Retrieved from <https://docs.geoserver.org/>
5. Leaflet. (2026). *Leaflet JS API Reference and Documentation*. Retrieved from <https://leafletjs.com/reference.html>

10.3 PAPER REFERENCES

1. Allafta & Opp (2021) – *GIS-based multi-criteria analysis for flood prone areas mapping in the trans-boundary Shatt Al-Arab basin, Iraq-Iran.*
2. Gupta & Dixit (2022) – *A GIS-based flood risk mapping of Assam, India, using the MCDA-AHP approach at the regional and administrative level.*
3. Bahramloo, Wang, Sepehri, Faghfour, Ghermezcheshmeh, Atapourfard, Talebi, Abbasi, & Bazrafshan (2024) – *Novel MCDA methods for flood hazard mapping: a case study in Hamadan, Iran.*
4. Kumar, Solanki, Sharma, Patel, Tiwari, & Mehta (2024) – *India's flood risk assessment and mapping with multi-criteria decision analysis and GIS integration.*
5. Puspitasari, Hizbaron, & Mei (2026) – *An urban flood-risk assessment of South Jakarta, Indonesia: A methodological approach through frequency ratio, receiver operating curve and analytic hierarchy process.*